

Lab 2: Visualization of global rates of cesarean delivery

Your name and student ID

August 27, 2023

Run the chunk below so the autograder package will load!

Instructions

- Due date: Friday, September 1st at 10:00pm PST with 2 hour grace period.
- Late penalty: 50% late penalty if submitted within 24 hours of due date, no marks for assignments submitted thereafter.
- This assignment is graded on **correct completion**, all or nothing. You must pass all public tests and submit the assignment for credit.
- Submission process: Follow the submission instructions on the final page. Make sure you do not remove any `\newpage` tags or rename this file, as this will break the submission.

Visualizing global cesarean delivery rates and GDP across 137 countries

The proportion of pregnant women who undergo cesarean deliveries varies dramatically across countries. In the US, the cesarean delivery rate is around 33%. A 2018 study in the British Medical Journal found that cesarean delivery rates ranged between 0.6% in South Sudan and 58.9% in the Dominican Republic.¹

Over the course of this lab and the first problem set, we will study the relationship between countries' cesarean delivery rates and their gross domestic products (GDPs).^{2,3}

In today's lab we will:

- import the data,
- familiarize ourselves with the dataset,
- explore the univariate distributions of the key variables.

If you haven't already, begin by knitting this document by clicking the "Knit" button above. As you fill out this lab, you can re-knit (click the button again) and see how the document changes.

Let's get started!

Click the green arrow icon in the top right corner of the gray chunk below to execute the three lines of code in the code chunk, or execute them line by line by placing your cursor on the first line and hitting `cmd + enter` on Mac or `ctrl + enter` on PC. If you do not want messages produced by loading `dplyr` to print in your knitted document, toggle `message = TRUE` to `message = FALSE` in the code chunk options. Re-knit your file and notice the difference in the output:

```
library(readr)
```

```
##
```

```
## Attaching package: 'readr'
```



```
## The following objects are masked from 'package:testthat':
##
##   edition_get, local_edition
```

```
library(dplyr)
```

```
##
## Attaching package: 'dplyr'
```

```
## The following object is masked from 'package:testthat':
##
##   matches
```

```
## The following objects are masked from 'package:stats':
##
##   filter, lag
```

```
## The following objects are masked from 'package:base':
##
##   intersect, setdiff, setequal, union
```

```
library(ggplot2)
```

- The `library` command loads the packages into memory.
- The `readr` library contains function for reading in data.
- The `dplyr` library contains functions we will use to manipulate data.
- The `ggplot2` library contains functions to visualize data.

The data on each country's cesarean delivery rate is stored in a comma-separated-values (csv) file. We can load that data and assign it to an object called `CS_data_raw` by running the code below. Click the green arrow in the line below to run the code. (Note that the green arrow will run all the code in this code chunk, not just the first line of code):

```
CS_data_raw <- read_csv("data/cesarean.csv")
```

```
## Rows: 137 Columns: 7
## -- Column specification -----
## Delimiter: ","
## chr (4): Country_Name, CountryCode, Income_Group, Region
## dbl (3): Births_Per_1000, GDP_2006, CS_rate
##
## i Use 'spec()' to retrieve the full column specification for this data.
## i Specify the column types or set 'show_col_types = FALSE' to quiet this message.
```

```
# The code below re-orders the variable Income_Group in the specified order.
# Note that it *does not* change the order of the data frame (like arrange() does)
# Rather, it specifies the order in which the data will be plotted.
# This will make more sense when we plot the data using Income_Group, and then again using Income_Group
```

```
CS_data_raw <- CS_data_raw %>%
  mutate(Income_Group_order = forcats::fct_relevel(Income_Group, "Low income",
                                                    "Lower middle income", "Upper middle income",
                                                    "High income: nonOECD", "High income: OECD"))
```

Look at the environment tab (top right pane) and make sure you see your dataset there. Click on `CS_data_raw` in that pane. This will open the data viewer tab. Take a quick look at the data and then switch back to the “lab02.Rmd” file and move to the next step.

When you receive a dataset you will also often receive a data dictionary that tells you what each variable means. Below is an example of a dictionary for this dataset:

Variable	Description
Country_Name	The name of the country
CountryCode	3-digit unique country identifier
Births_Per_1000	The number of births per 1000 people in the country
Income_Group	The income group the country is categorized into
Region	The geographic region the country belongs to
GDP_2006	The gross domestic product (GDP) of the country in 2006
CS_rate	The percent of births that were delivered by cesarean section
Income_Group_order	The “ordered” version of Income_Group

Look at your newly imported data

In four separate code chunks, type `str(CS_data_raw)`, `dim(CS_data_raw)`, `names(CS_data_raw)`, and `head(CS_data_raw)`. Execute each code chunk and study the output.

```
str(CS_data_raw)
```

```
## tibble [137 x 8] (S3: tbl_df/tbl/data.frame)
## $ Country_Name      : chr [1:137] "Albania" "Andorra" "United Arab Emirates" "Argentina" ...
## $ CountryCode       : chr [1:137] "ALB" "AND" "ARE" "ARG" ...
## $ Births_Per_1000   : num [1:137] 46 1 63 689 47 267 76 166 119 342 ...
## $ Income_Group      : chr [1:137] "Upper middle income" "High income: nonOECD" "High income: nonOECD" ...
## $ Region            : chr [1:137] "Europe & Central Asia" "Europe & Central Asia" "Middle East & North Africa" ...
## $ GDP_2006          : num [1:137] 3052 42417 42950 6649 2127 ...
## $ CS_rate           : num [1:137] 0.256 0.237 0.1 0.352 0.141 0.303 0.271 0.076 0.159 0.036 ...
## $ Income_Group_order: Factor w/ 5 levels "Low income","Lower middle income",...: 3 4 4 4 2 5 5 3 5 1
```

```
dim(CS_data_raw)
```

```
## [1] 137 8
```

```
names(CS_data_raw)
```

```
## [1] "Country_Name"      "CountryCode"      "Births_Per_1000"
## [4] "Income_Group"      "Region"           "GDP_2006"
## [7] "CS_rate"           "Income_Group_order"
```

```
head(CS_data_raw)
```

```
## # A tibble: 6 x 8
##   Country_Name CountryCode Births_Per_1000 Income_Group Region GDP_2006 CS_rate
##   <chr>         <chr>          <dbl> <chr>      <chr>      <dbl>   <dbl>
## 1 Albania      ALB              46 Upper middl~ Europ~    3052.   0.256
## 2 Andorra      AND               1 High income~ Europ~   42417.   0.237
## 3 United Arab ~ ARE              63 High income~ Middl~   42950.   0.1
## 4 Argentina    ARG             689 High income~ Latin~    6649.   0.352
## 5 Armenia      ARM              47 Lower middl~ Europ~    2127.   0.141
## 6 Australia    AUS             267 High income~ East ~    36101.   0.303
## # i 1 more variable: Income_Group_order <fct>
```

1. In your own words, what do these functions do?

- `dim()`: returns the number of rows/individuals and number of columns/variables.
- `str()`: returns information about the types of variables found in the dataset in terms of whether they are quantitative (int, num) or categorical (factor) and provides some information about each one.
- `names()`: returns the variable names.
- `head()`: returns the first six rows of the dataset.

2. Based on these functions' outputs, who/what are the individuals in this dataset? How many of them are there?

The individuals are countries. There are 137 of them.

Add a new variable

3. [3 points] Right now, `CS_rate` is presented as a number between 0 and 1. For plotting, it will be nice to have the proportion displayed as a number between 0 and 100. We can accomplish this using `dplyr`'s `mutate()` function to add a new variable called `CS_rate_100`. Write one line of code to add this variable to the dataset and assign this new dataframe to an object called `CS_data`.

```
CS_data <- CS_data_raw %>% mutate(CS_rate_100 = CS_rate*100)
CS_data
```

```
## # A tibble: 137 x 9
##   Country_Name CountryCode Births_Per_1000 Income_Group Region GDP_2006 CS_rate
##   <chr>         <chr>          <dbl> <chr>         <chr>    <dbl>   <dbl>
## 1 Albania      ALB              46 Upper middl~ Europ~   3052.   0.256
## 2 Andorra      AND               1 High income~ Europ~  42417.   0.237
## 3 United Arab~ ARE              63 High income~ Middl~  42950.   0.1
## 4 Argentina    ARG             689 High income~ Latin~   6649.   0.352
## 5 Armenia      ARM              47 Lower middl~ Europ~   2127.   0.141
## 6 Australia    AUS             267 High income~ East ~   36101.   0.303
## 7 Austria      AUT              76 High income~ Europ~  40431.   0.271
## 8 Azerbaijan   AZE             166 Upper middl~ Europ~   2473.   0.076
## 9 Belgium      BEL             119 High income~ Europ~  38936.   0.159
## 10 Benin       BEN             342 Low income  Sub-S~    557.   0.036
## # i 127 more rows
## # i 2 more variables: Income_Group_order <fct>, CS_rate_100 <dbl>
```

```
# YOUR CODE HERE
```

```
. = ottr:::check("tests/p3.R")
```

```
##
## All tests passed!
```

Examine the distribution of income and geographic regions

The countries in the dataset represent a wide range of income levels and geographic regions.

4. Which variable groups the countries by income regions?

'Income_Group'

5. Which variable groups the countries by geographic regions?

'Region'

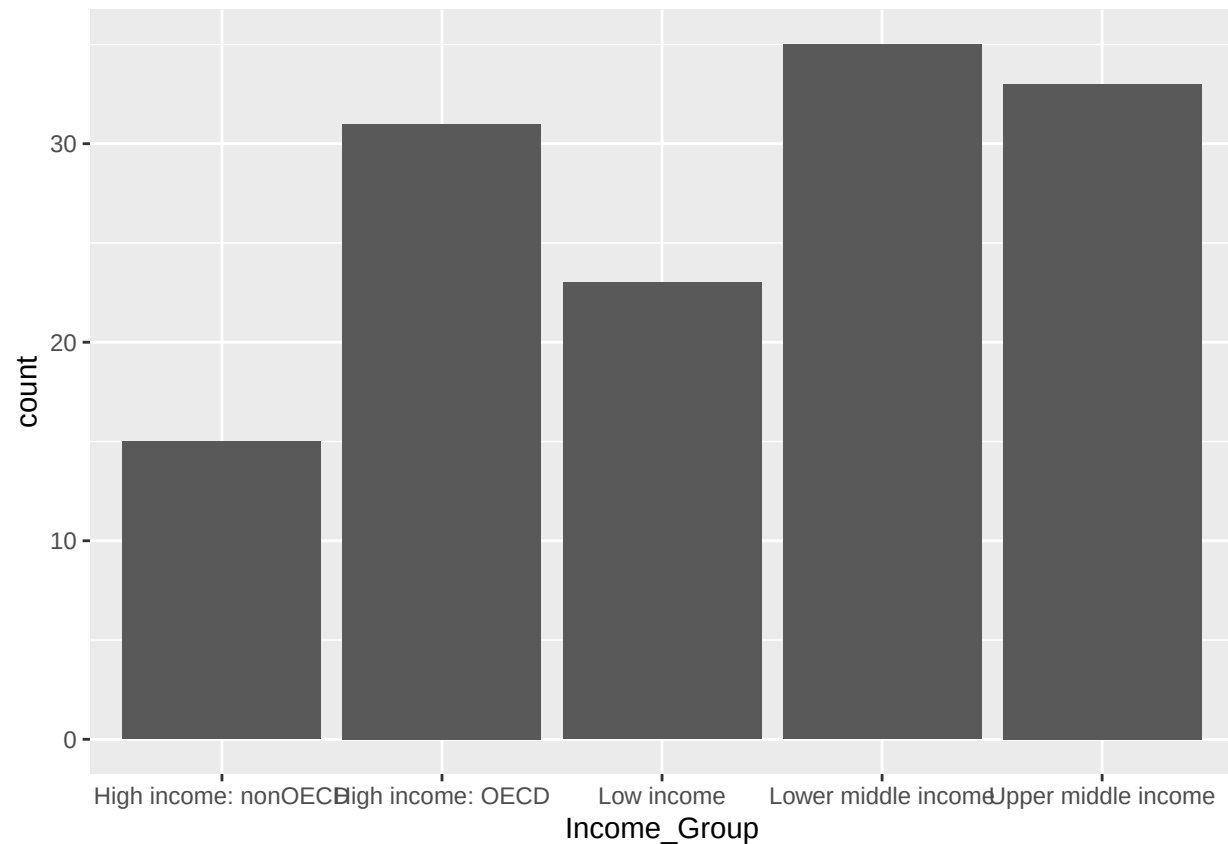
These variables are categorical variables. So far in lecture, we've seen categorical variables stored as `chr` variables when we've examined `str(data)`. However, `Income_Group_order` is stored as a `Factor` variable in this dataset, which is another way to store categorical data. One benefit of storing it as `Factor` is that we can re-order the `Factor` variable in a way that makes sense for plotting and analysis. To see the values and order of a factor variable, we can use the following code.

STOP: Remove `eval = F` from this chunk header before continuing on.

```
CS_data %>% pull(Income_Group_order) %>% levels  
# You won't be tested on writing the line of code above.  
# Try and understand what the code is showing you.
```

6. [1 point] Because Income_Group and Region are categorical, we make bar charts to represent their distributions. Using the `ggplot()` code you learned during lecture, make a bar chart showing the number of countries per income group using `Income_Group` and assign this plot to `p6`.

```
p6 <- ggplot(CS_data, aes(x = Income_Group)) + geom_bar()  
p6
```



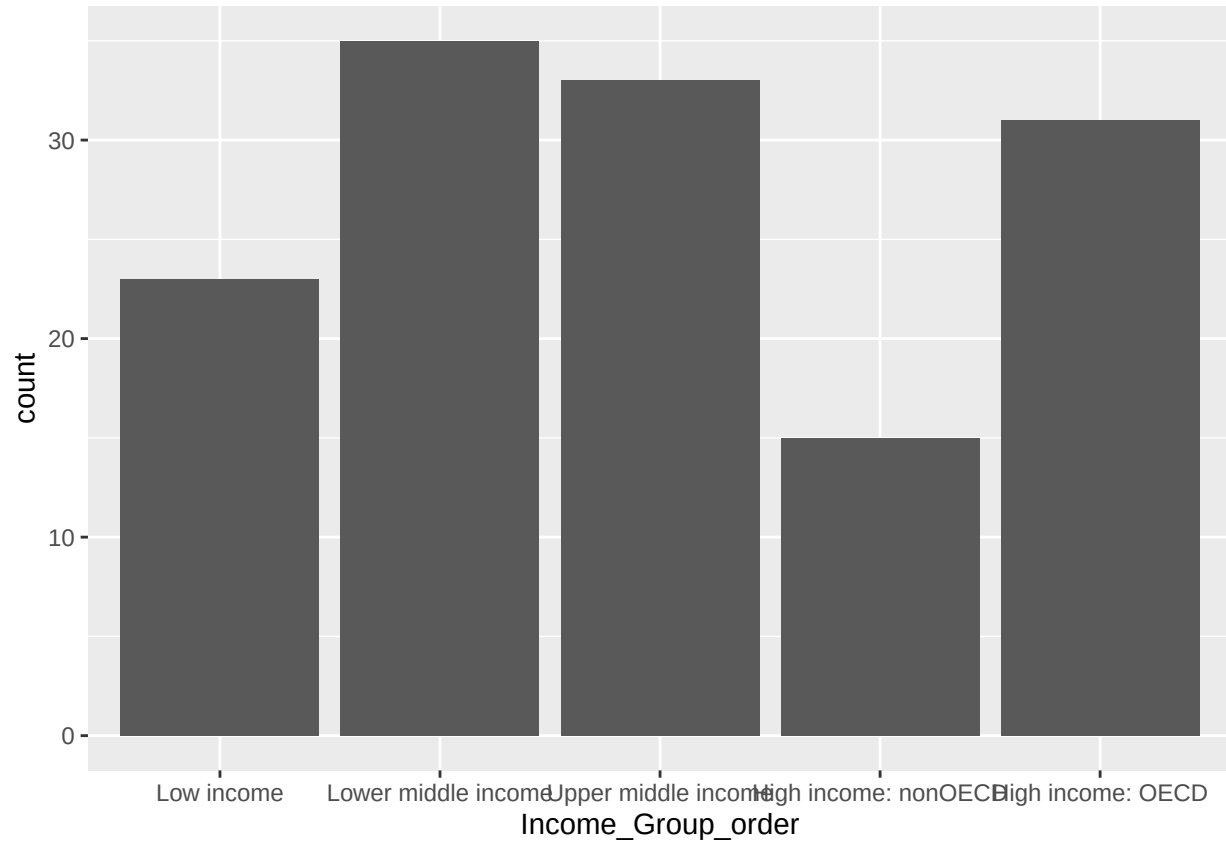
```
# YOUR CODE HERE
```

```
. = ottr::check("tests/p6.R")
```

```
##  
## All tests passed!
```

7. [1 point] Now make the bar chart using `Income_Group_order`.

```
p7 <- ggplot(CS_data, aes(x = Income_Group_order)) + geom_bar()
p7
```



```
# YOUR CODE HERE
```

```
. = ottr::check("tests/p7.R")
```

```
##
```

```
## All tests passed!
```

8. Do you prefer the plot using `Income_Group` or `Income_Group_order`? Why?

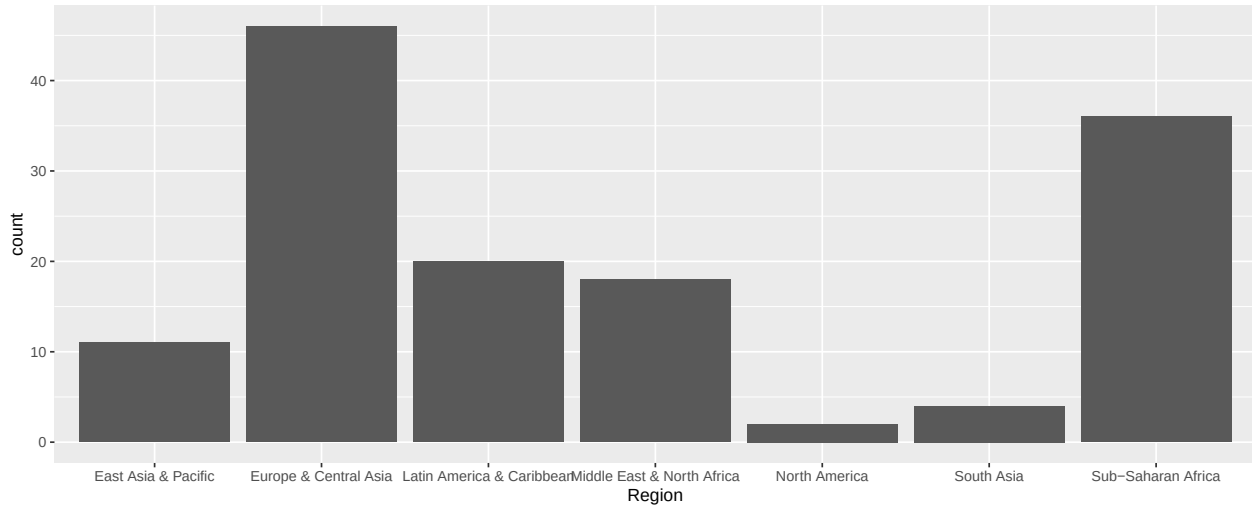
`Income_Group_order` is preferred because the visualization is ordered by increasing levels of income, while using `Income_Group` displays the income in alphabetical order: high income is first, followed by low income, and then middle income, which is less intuitive.

9. Did you use the argument `stat="identity"` in the code to make this bar chart? Why or why not?

No, I didn't use `stat="identity"` argument. This is because I did not specify a y variable containing a count or percent for ggplot to plot on the y-axis. When `geom_bar()` is doing the counting for us, we do not use the `stat="identity"` argument.

10. [1 point] Make a bar chart showing the number of countries per geographic region.

```
p10 <- ggplot(CS_data, aes(x = Region)) + geom_bar()
p10
```



```
# YOUR CODE HERE
```

```
. = ottr::check("tests/p10.R")
```

```
##
```

```
## All tests passed!
```

11. Based on your plot, which world region has the most countries in the dataset?

Europe & Central Asia

12. Which geographic region has the fewest number of countries in the dataset?

North America

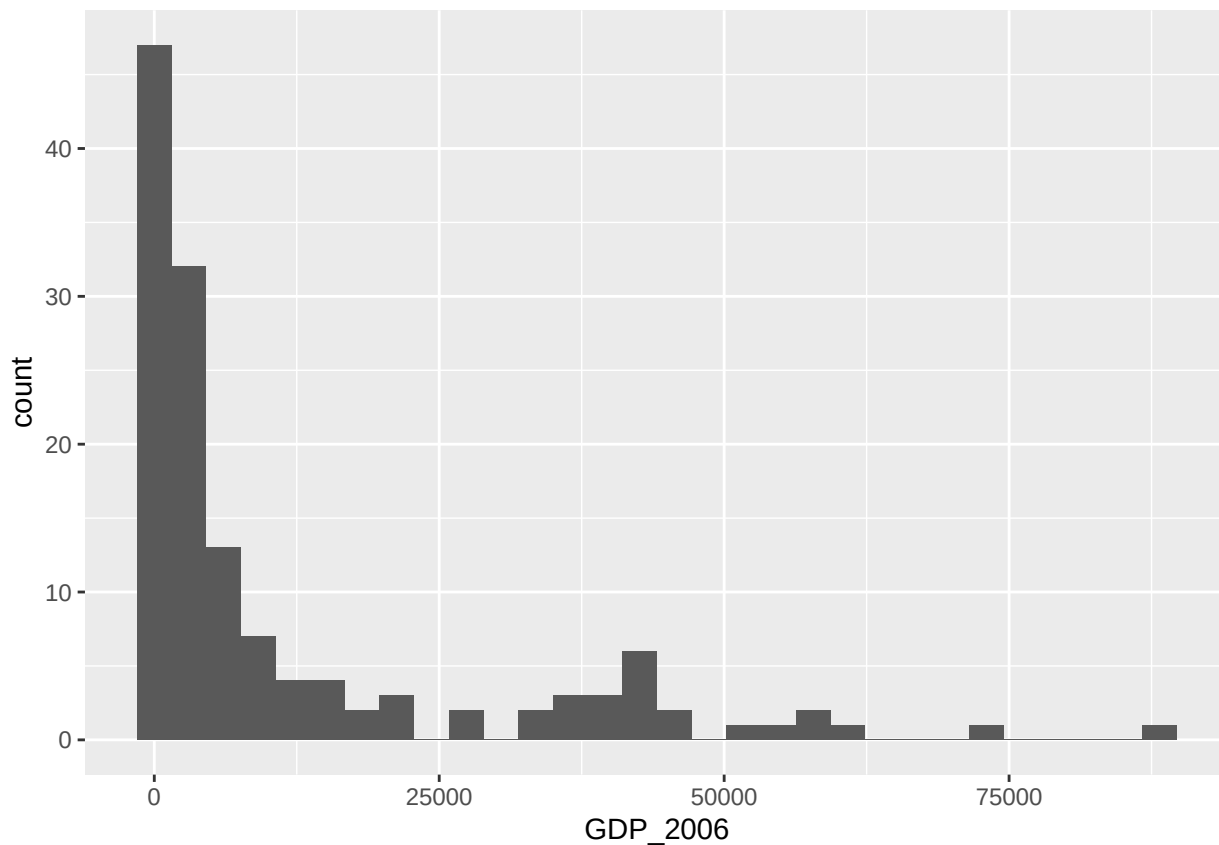
Try changing the `fig.width`! Note that the code chunks where you made your plots had the chunk option “`fig.width = 11`”. Try changing the setting to a lower number, like 9, and re-running the chunk. Notice how the width of the rendered plot changes. Change it back to 11 so that the x-axis labels don’t look too cramped.

Examine the distribution of `GDP_2006` and `CS_rate_100` across countries

13. [1 point] `GDP_2006` and `CS_rate_100` are quantitative, meaning that we use histograms rather than bar charts to examine their distributions. Using code you learned in class, make a histogram of `GDP_2006`. Pay attention to the message that R outputs when running your code. It is telling you that by default the data is grouped into 30 bins for plotting.

```
p13 <- ggplot(CS_data, aes(x = GDP_2006)) + geom_histogram()
p13
```

```
## 'stat_bin()' using 'bins = 30'. Pick better value with 'binwidth'.
```



```
# YOUR CODE HERE
```

```
. = ottr::check("tests/p13.R")
```

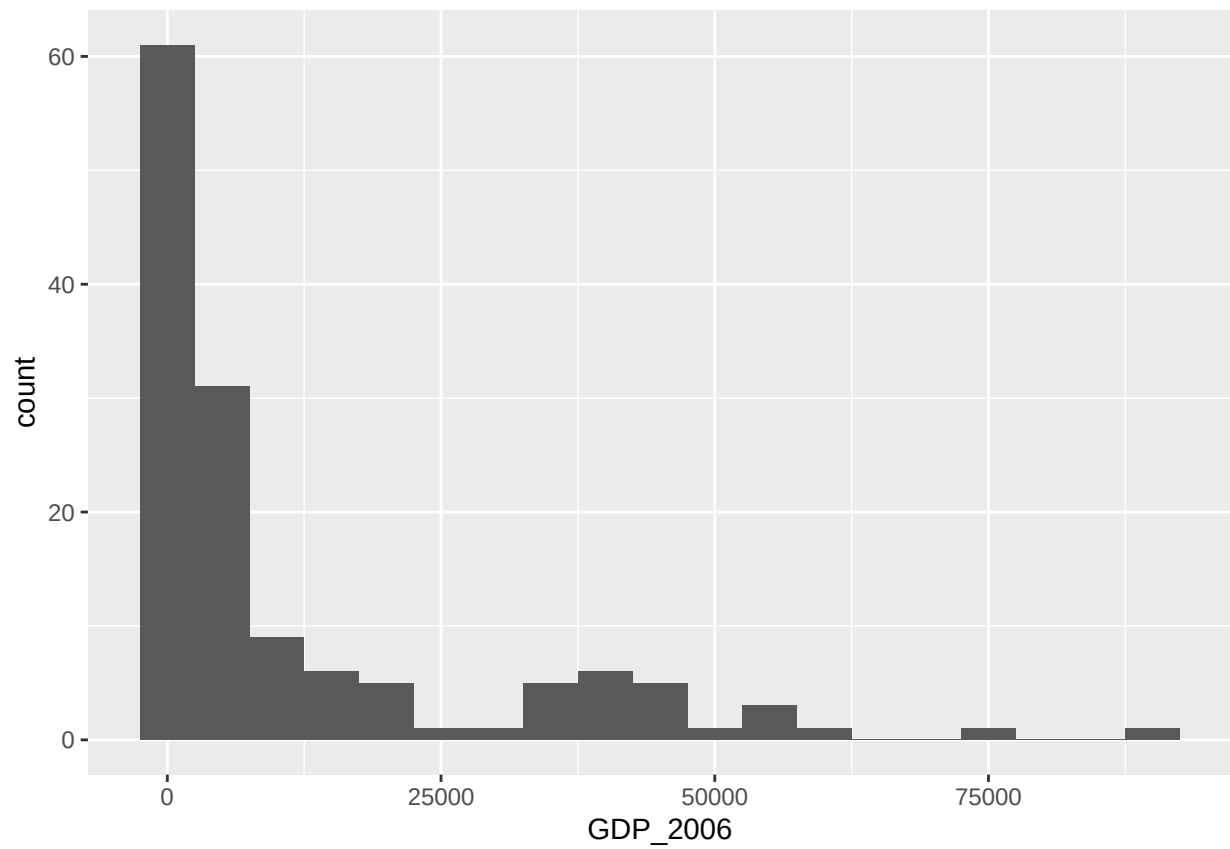
```
##
```

```
## All tests passed!
```

14. [1 point] Rather than letting R choose the number of bins, we can set `binwidth = number` within the `geom` function to choose the width of the bins. Try setting the binwidth to different values and see how it changes the plot.

To choose a binwidth, look at the x-axis of the histogram you just made. Because we are plotting GDP across a wide variety of countries, it might make sense to choose a binwidth in the 1000s. Thus, the range of the variable matters when choosing a good binwidth.

```
p14 <- ggplot(CS_data, aes(x = GDP_2006)) + geom_histogram(binwidth = 5000)
p14
```



```
# YOUR CODE HERE
```

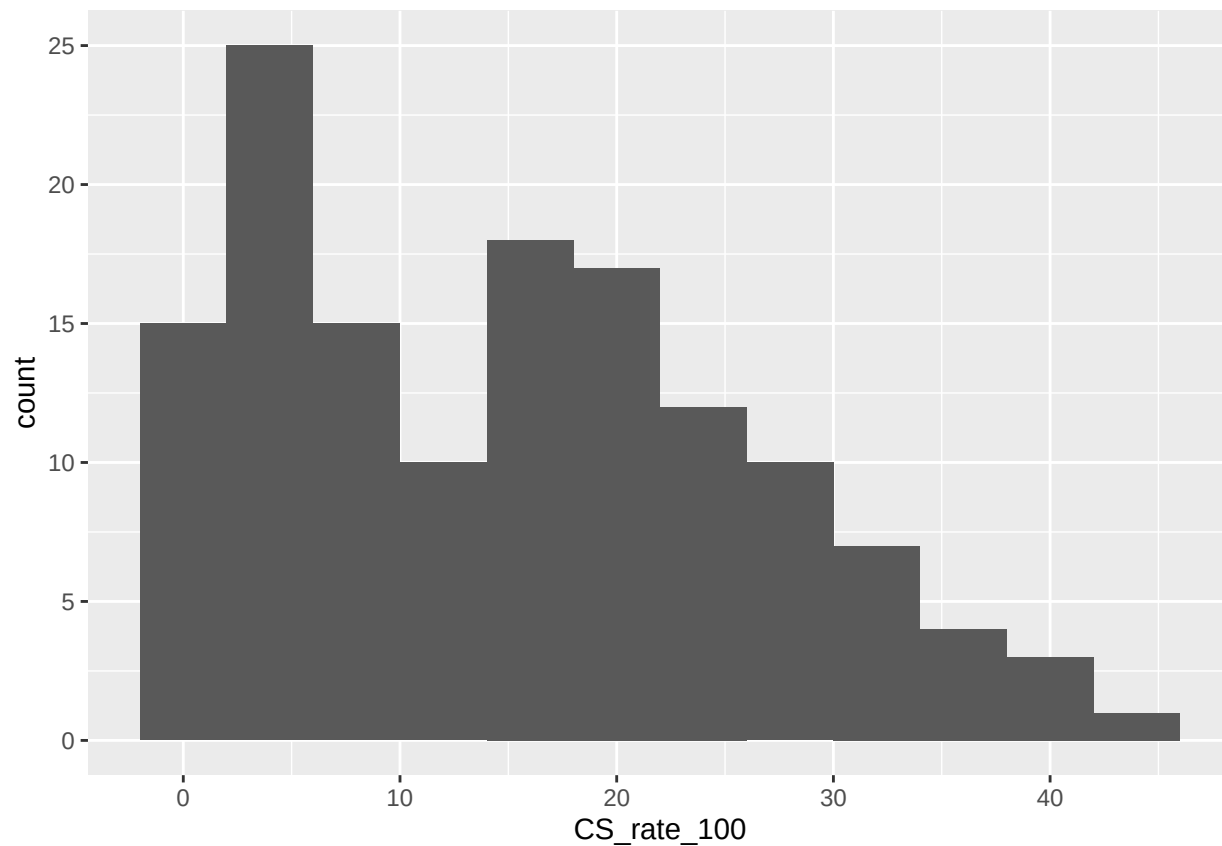
```
. = ottr::check("tests/p14.R")
```

```
##
```

```
## All tests passed!
```

15. [1 point] Next, make a histogram for cesarean delivery rates using the variable `CS_rate_100`. First run your code without a set binwidth and then add the `binwidth` argument with an appropriate value.

```
p15 <- ggplot(CS_data, aes(x = CS_rate_100)) + geom_histogram(binwidth = 4)
p15
```



```
# YOUR CODE HERE
```

```
. = ottr::check("tests/p15.R")
```

```
##
```

```
## All tests passed!
```

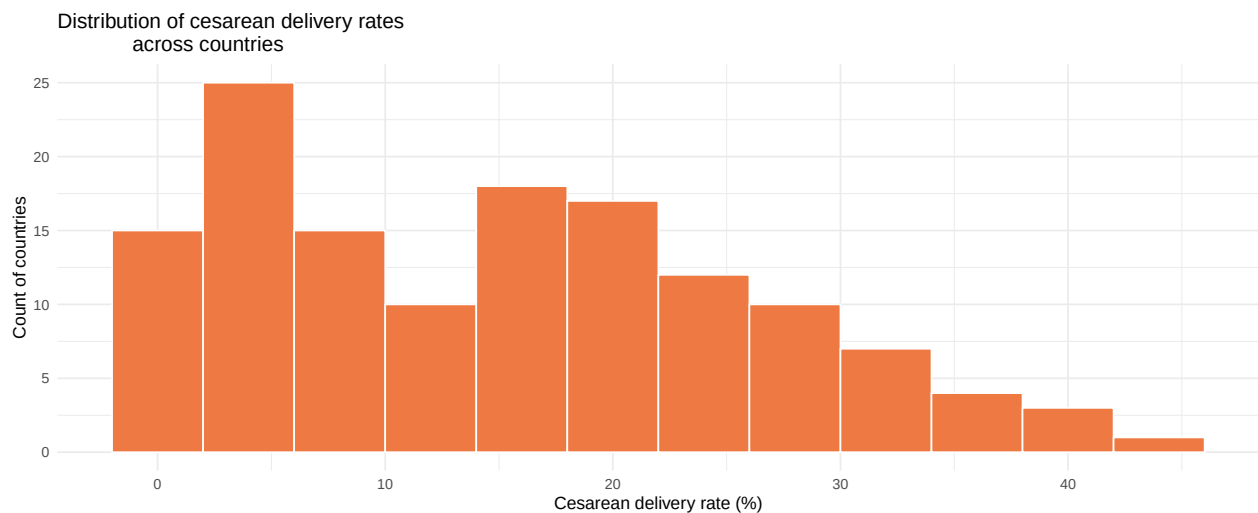
Polish up your plots

Your plots look great so far, but let's get them ready to present at a meeting or in a report. To do this, please update the plot and axes titles (using `labs`), and change the fill or color of the bars. Here is a big list of colors that you can use to change the `col` and `fill`. For example `col = "orchid1"` will change the color to a pinky-purple.

16. [1 point] Present your final plots with updated titles, labels, colors, and fills below. To do this, extend the ggplot code you wrote above with the functions and arguments just described.

```
p16 <- ggplot(CS_data, aes(x = CS_rate_100)) +  
  geom_histogram(binwidth = 4, col = "white", fill = "sienna2") +  
  labs(y = "Count of countries", x = "Cesarean delivery rate (%)",  
       title = "Distribution of cesarean delivery rates  
               across countries") +  
  theme_minimal()
```

p16



```
# YOUR CODE HERE
```

```
. = ottr::check("tests/p16.R")
```

```
##
```

```
## All tests passed!
```

Save your plots to separate files

Sometimes you need to save plots as a PNG or JPEG to include in a PowerPoint presentation or submit to a research journal for publication. The `ggsave()` function is used to save plots as separate files. Quickly read the documentation of the `ggsave()` function by executing this code which opens a help window in the bottom right pane:

```
?ggsave
```

17. Use the documentation to attempt saving your plots in the code chunk below. Don't worry about producing an error or saving your plot in the wrong place – that is all part of coding and learning new functions!

NOTE: the `filename` argument cannot be “lab02.Rmd”! This is the name of the lab file you are working in, so if you save the image of your plot as lab02.Rmd it will override all of the hard work you’ve done in this lab. Instead, call your image something like “CS_plot.png” and specify the `device` to be “png”.

```
ggsave(filename = "hist-cs.png", plot = p16,  
        device = "png", width = 9, height = 5, units = "in")  
# Notice that the plot resolution is set using `dpi = 720` here which  
# is often in the range of dpis requested when submitting figures for  
# publication as part of a journal article.  
# YOUR CODE HERE
```

When you are done, knit your R document one last time and ensure it looks good! Change anything you don't like. You can choose to knit to .docx or .pdf by selecting these options from the dropdown menu next to knit (in case you'd like to save the completed file onto your computer).

When you're happy with your work, please follow the instructions on the final page to submit.

References

1. Boatin AA, Schlottheuber A, Betran AP, Moller AB, Barros AJ, Boerma T, Torloni MR, Victora CG, Hosseinpour AR. Within country inequalities in caesarean section rates: observational study of 72 low and middle income countries. *BMJ*. 2018;24(360):k55. DOI: 10.1136/bmj.k55.
2. Gibbons L, Belizán JM, Lauer JA, Betrán AP, Merialdi M, Althabe F. The global numbers and costs of additionally needed and unnecessary caesarean sections performed per year: overuse as a barrier to universal coverage. World Health Organization; 2010. Available at: <http://www.who.int/healthsystems/topics/financing/healthreport/30C-sectioncosts.pdf>. Accessed October 4, 2015.
3. The World Bank. GDP per capita (current US\$). Washington, DC; Available at: <http://data.worldbank.org/indicator/NY.GDP.PCAP.CD>. Accessed Sep 30, 2015.

Submission

For assignments in this class, you'll be submitting using the **Terminal** tab in the pane below. In order for the submission to work properly, make sure that:

1. Any image files you add that are needed to knit the file are in the **src** folder and file paths are specified accordingly.
2. You **have not changed the file name** of the assignment.
3. The file knits properly.

Once you have checked these items, you can proceed to submit your assignment.

1. Click on the **Terminal** tab in the pane below.
2. Copy-paste the following line of code into the terminal and press enter.

```
cd; cd ph142-fa23/lab/lab02; python3 turn_in.py
```

3. Follow the prompts to enter your Gradescope username and password.
4. If the submission is successful, you should see "Submission successful!" appear as the output. **Check your submission on the Gradescope website to ensure that the autograder worked properly and you received credit for your correct answers. If you think the autograder is incorrectly grading your work, please post on piazza!**
5. If the submission fails, try to diagnose the issue using the error messages—if you have problems, post on Ed under the post "Datahub Issues".

The late policy will be strictly enforced, **no matter the reason**, including submission issues, so be sure to submit early enough to have time to diagnose issues if problems arise.